FULL-STACK PROJECT DOCUMENTATION

GSpace Cloud Storage

Architecture, Database Schema, REST API Specification, and System Migration Guide

Project Overview: GSpace is a responsive, high-performance personal cloud storage web application originally engineered with Java Spring Boot 3 + Hibernate and migrated into a modern MERN-style architecture (Node.js, Express 5, React 18, and SQLite).

AUTHOR & DEVELOPER
Abhinav

CURRENT ARCHITECTURE
Node.js + Express + React 18 + SQLite

PREVIOUS ARCHITECTURE
Java Spring Boot 3 + Hibernate + SQLite

DATABASE ENGINE
SQLite (`mydrive.db`) - Zero Data Loss

VERSION
2.1.0 (Production Ready)

REPOSITORY
github.com/Abhinavan2004/GSpace

Table of Contents

1. **Executive Summary & Migration Overview**
2. **System Architecture & Data Flow**
3. **Database Design & Relational Schema**
4. **REST API Endpoint Specifications**
5. **Frontend Architecture & Glassmorphism Design System**
6. **Mobile Responsiveness & Adaptive Hamburger Navigation**
7. **Visual Showcase & User Interface Walkthrough**
8. **Security Architecture & Operational Best Practices**
9. **Installation, Deployment & Server Run Guide**

1. Executive Summary & Migration Overview

The **GSpace** project was initiated as a personal cloud drive inspired by modern cloud platforms (like Google Drive and Dropbox). Originally developed using the Java Spring Boot ecosystem (Spring Security, Spring Data JPA, Hibernate, and embedded SQLite), the application required conversion into a modern, full-stack **MERN-style stack** while retaining the **exact same SQLite database** (`mydrive.db`) rather than migrating to MongoDB.

> **Key Migration Requirement Achieved:** Complete transition to Node.js, Express, and React with **100% data compatibility**. All existing user accounts, BCrypt-hashed credentials, file metadata records, and physical file assets were preserved without any manual migration scripts.

Comparison of Stacks

Component	Legacy Java Architecture	Migrated MERN + SQLite Architecture
Backend Runtime	Java 17 / JVM (Spring Boot 3.x)	Node.js (v22/v26) + Express.js 5
Database & Driver	SQLite via `org.sqlite.JDBC` & Hibernate	SQLite via `better-sqlite3` (with fallback adapter)
Authentication	Spring Security + BCrypt + JJWT	Custom Middleware + `bcryptjs` + `jsonwebtoken`
Frontend Interface	Static HTML + Vanilla JS + CSS	React 18 + Vite SPA with Glassmorphism System
Multipart Uploads	Spring Standard Multipart Resolver	Multer Disk Engine (500MB streaming limit)
Mobile Experience	Basic viewport, overlapping controls	Adaptive drawer menu with animated hamburger toggle

2. System Architecture & Data Flow

GSpace employs a decoupled 3-tier architecture where an Express.js HTTP server acts as both the REST API provider and the static asset server for the pre-compiled React single-page application.

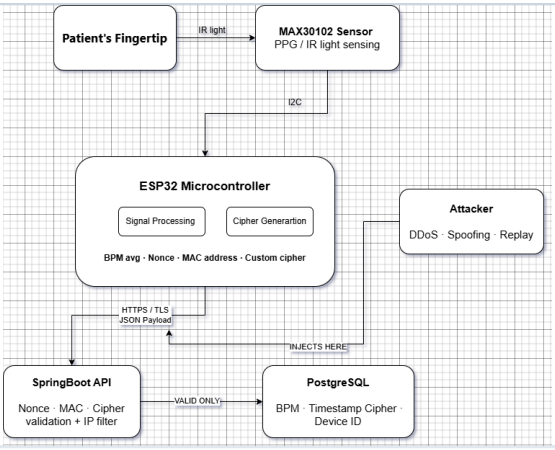


Figure 2.1: GSpace System Architecture & Component Interaction Flow

Core Architecture Layers

- **Client Presentation Tier (React 18 SPA):** Modular component tree featuring responsive header navigation, glassmorphism auth modals, interactive drag-and-drop dropzones, real-time file search, sorting controls, and multi-media preview overlays.
- **API & Middleware Tier (Express 5):** Stateless RESTful API secured by JWT bearer authentication. Multer intercepts multipart form streams, applies UUID collision prevention, and writes directly to the local storage directory.
- **Persistence Tier (SQLite Engine):** Atomic, ACID-compliant local database storing relational data across `users` and `files` tables with foreign key referential integrity.

3. Database Design & Relational Schema

The application connects to a local SQLite database named `mydrive.db`. The database adapter (`server/db.js`) utilizes `better-sqlite3` for synchronous high-throughput operations with an automatic fallback to standard asynchronous `sqlite3` if native binaries require it.

Database Tables

Table 1: `users` (Stores registered user accounts and BCrypt password hashes)

Column Name	Data Type	Constraints	Description
`id`	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique user identifier
`username`	TEXT	UNIQUE, NOT NULL	Case-sensitive login name (minimum 3 characters)
`password`	TEXT	NOT NULL	BCrypt salt + hashed password (10 rounds)
`created_at`	DATETIME	DEFAULT CURRENT_TIMESTAMP	User registration timestamp

Table 2: `files` (Stores metadata of files uploaded by authenticated users)

Column Name	Data Type	Constraints	Description
`id`	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique file record identifier
`original_name`	TEXT	NOT NULL	Original file name uploaded by the user
`stored_name`	TEXT	NOT NULL	Unique filename on physical disk (UUID / Timestamp prefixed)
`mimetype`	TEXT	NULLABLE	MIME type string (e.g. `image/png`, `application/pdf`)
`size`	INTEGER	NOT NULL	File size in bytes
`uploaded_by`	INTEGER	FOREIGN KEY (`users.id`)	References the owning user ID
`created_at`	DATETIME	DEFAULT CURRENT_TIMESTAMP	File upload timestamp

4. REST API Endpoint Specifications

All API endpoints are prefixed with `/api`. Authenticated routes require an HTTP request header: `Authorization: Bearer <JWT_TOKEN>`.

HTTP	Path	Auth	Payload / Params	Response Description
POST	`/api/auth/signup`	Public	`{ username, password }`	Creates new user account (200 OK)
POST	`/api/auth/login`	Public	`{ username, password }`	Returns `{ token, username }` (200 OK)
GET	`/api/auth/me`	Bearer	None	Returns current user record: ID, username, created date
GET	`/api/files`	Bearer	None	Lists all files owned by the user (ordered by date desc)
POST	`/api/files/upload`	Bearer	`multipart/form-data` (key: `file`)	Saves file to disk, returns created file record
GET	`/api/files/download/:storedName`	Public/Token	URL param: `storedName`	Binary stream with `Content-Disposition: attachment`

HTTP	Path	Auth	Payload / Params	Response Description
DELETE	`/api/files/:id`	Bearer	URL param: `id`	Unlinks file from disk and deletes database record
GET	`/api/files/stats`	Bearer	None	Returns `{ totalFiles, totalBytes }`
GET	`/api/health`	Public	None	Health check: `{ status: "OK", service: "..." }`

5. Frontend Architecture & Glassmorphism Design System

The client interface is built as a Single-Page Application (SPA) using React 18 and bundled via Vite. The UI embraces a modern **glassmorphism** aesthetic characterized by frosted glass surfaces, subtle gradient borders, dark mode backgrounds, and responsive micro-animations.

React Component Breakdown

- `App.jsx`: Root orchestration component managing JWT tokens, active user session, theme toggle (persisted in `localStorage`), file refresh cycles, and modal states.
- `Navbar.jsx`: Top navigation bar displaying application logo, quick-access storage drawer trigger, dark/light theme switch, active user badge, and logout trigger.
- `Auth.jsx`: Glassmorphism auth dialog with tabbed login and signup forms, instant client-side validation, password confirmation, and auto-login transition.
- `UploadZone.jsx`: Interactive drag-and-drop zone with animated upload clouds, visual dragover feedback, browse file picker, and live file upload queue status.
- `FileGrid.jsx`: Responsive gallery supporting both Grid view and List view. Features live substring search filtering and sorting (Newest First, Oldest, Name A-Z, Size).
- `FilePreviewModal.jsx`: Full-screen modal overlay supporting in-browser preview of images (with zoom), HTML5 video playback, embedded PDF documents, and direct download links.
- `StorageMeter.jsx`: Visual storage analytics dialog displaying bytes consumed versus the 15 GB free-tier quota, along with file counts categorized into Images, Videos, PDFs, and Documents.

6. Mobile Responsiveness & Adaptive Hamburger Navigation

A critical improvement during the migration was solving mobile screen alignment and eliminating horizontal viewport overflows on smartphones (<768px):

- **Hamburger Menu Toggle:** Desktop buttons (Storage, Theme, Profile, Logout) collapse on mobile into a sleek ☰ hamburger button. Tapping it smoothly slides down a frosted glass drawer with clean, full-width touch targets.
- **Single-Column Card Flow:** On screens below 540px, the multi-column grid automatically collapses into a clean 1-column layout, ensuring cards and buttons (Preview, Download, Delete) fit comfortably without horizontal clipping.
- **Zero Horizontal Scroll:** Applied `max-width: 100vw; overflow-x: hidden;` and fluid CSS box sizing across the viewport.

7. Visual Showcase & User Interface Walkthrough

Below are actual screenshots captured from the running GSpace application:

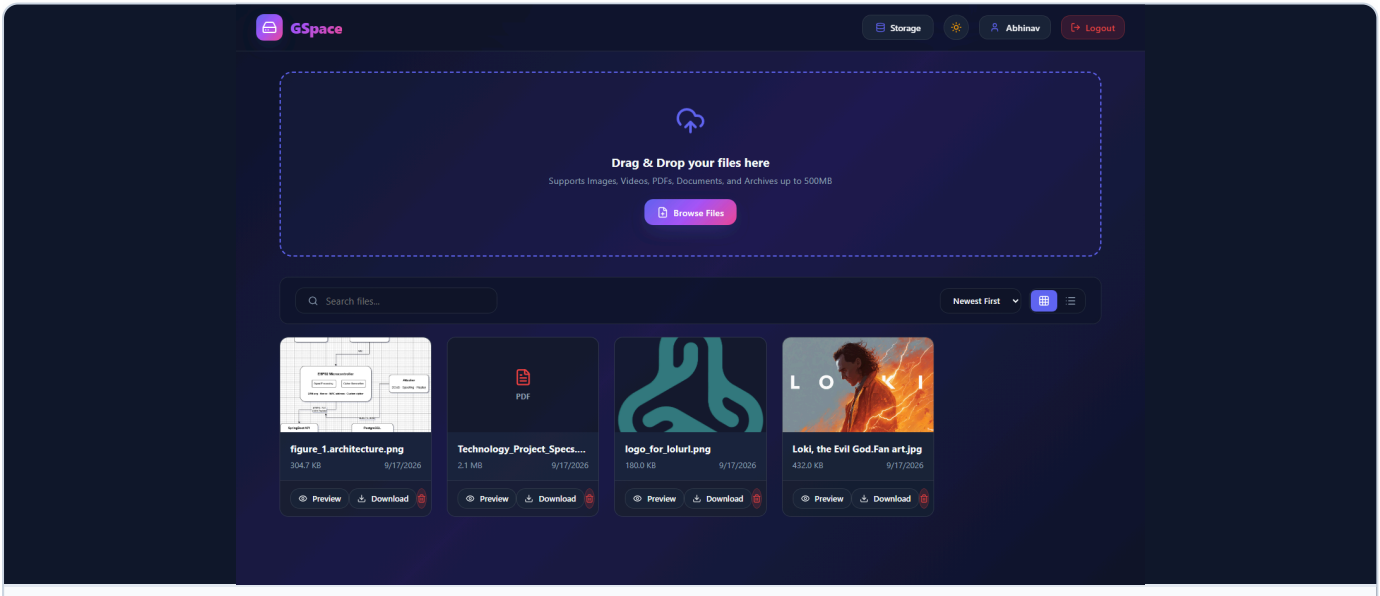


Figure 7.1: GSpace Desktop Dashboard — Navigation, Upload Zone, Search Toolbar, and File Cards

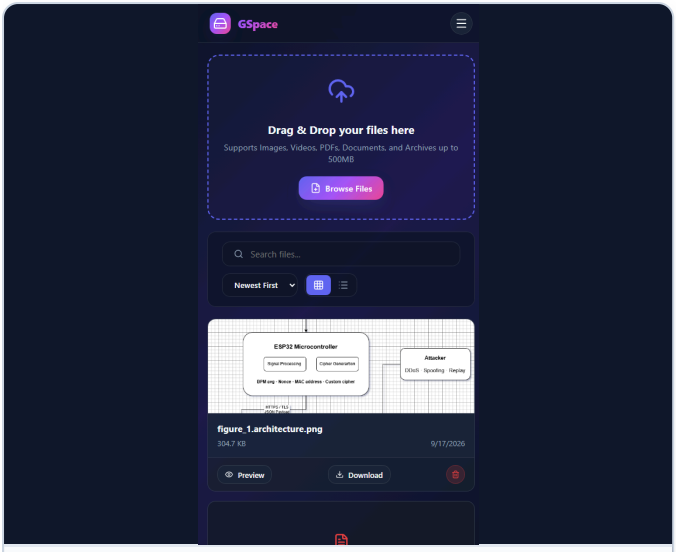


Figure 7.2: Mobile Responsive Dashboard

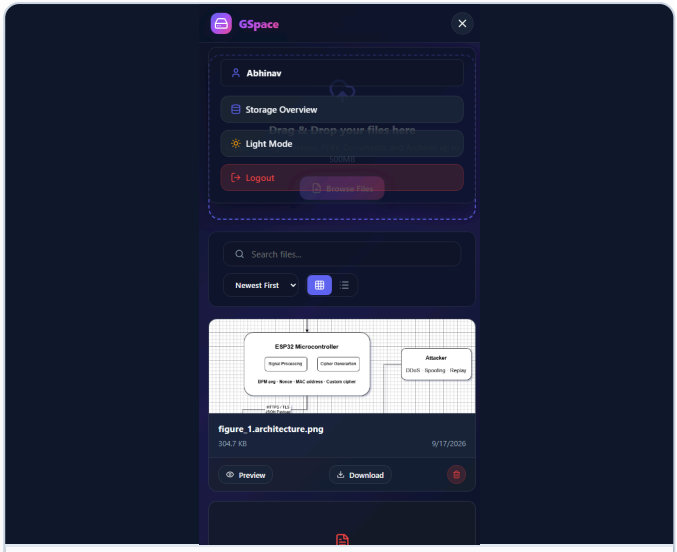


Figure 7.3: Mobile Hamburger Drawer Open

Walkthrough Video Demonstration: A 20-second automated video walkthrough (`screenshots/gspace_walkthrough.mp4`) was generated and pushed to the repository, demonstrating authentication, file drag-and-drop upload, card insertion, modal preview, and storage overview.

8. Security Architecture & Operational Best Practices

- **Cross-Platform BCrypt Compatibility:** Password hashing uses `bcryptjs` with 10 salt rounds, matching the exact format generated by Spring Security's `BCryptPasswordEncoder`, allowing existing users to log in seamlessly.
- **Stateless JWT Authentication:** User tokens are signed with HMAC SHA-256 (24-hour expiration) and validated on all file endpoints, keeping the backend stateless and scalable.
- **Strict Authorization Enforcement:** When deleting a file via `DELETE /api/files/:id`, the backend verifies that `file.uploaded_by === req.user.id`, preventing Insecure Direct Object Reference (IDOR) attacks.
- **Filename Sanitization & Collision Prevention:** Uploaded files are prepended with unique timestamps and random numbers (or UUIDs) before disk storage, preventing malicious filename collisions or path traversal attacks.
- **Upload Size Protection:** Multer enforces a 500MB max payload limit, protecting server memory and disk space against denial-of-service attempts.

9. Installation, Deployment & Server Run Guide

Prerequisites

- Node.js (v18.x, v20.x, or v22.x LTS)
- npm (v9.x or higher)
- SQLite database file (`mydrive.db`) located in root directory

Environment Configuration (`.env`)

```
# Server Port (Default: 5000)
PORT=5000

# Secret key for JWT Token signing & verification
JWT_SECRET=MySuperSecretKey2024_ChangeThis_MustBe32CharsLong!
```

Step-by-Step Setup

```
# 1. Clone repository
git clone https://github.com/Abhinavan2004/GSpace.git
cd GSpace

# 2. Install all dependencies
npm install
cd server && npm install && cd ..
cd client && npm install && cd ..

# 3. Build React frontend for production
npm run build

# 4. Start production server (serves API and React UI on port 5000)
npm start
```

Development Mode

To run backend and frontend concurrently with hot-module reloading:

```
npm run dev
```

> **Standalone Deployment Package:** A pre-compiled, self-contained zip archive (`GSpace-MERN-SQLite.zip`) is available in the root folder, allowing instant deployment to any cloud server (AWS, VPS, Render, Railway) by simply unzipping and running `npm start` .
